

01 / DISCOVER

Start With the Problem, Not the Feature

Discovery is the discipline of understanding what is really happening before a team commits time, money and attention to a solution.

4 MIN READ

“A request is a signal. It is not automatically the problem.”

Discovery begins before the roadmap

Some of the most expensive product mistakes begin with a sentence that sounds completely reasonable: ‘Users need this feature.’ The request may come from a customer, a stakeholder, support, sales or even someone inside the product team. It is tempting to open the backlog, create a ticket and start discussing how quickly it can be shipped.

I try not to start there. I want to know what is happening underneath the request. What is the user trying to achieve? Where are they getting stuck? Is this happening to one person or a meaningful group? Is the pain caused by the absence of a feature, or by something already in the product that is difficult to understand or use?

That is what discovery means to me. It is not a ceremony that only happens at the beginning of a new product. It is the habit of slowing down just enough to understand the problem before accelerating into delivery.

What I am really looking for

During discovery, I am trying to reduce uncertainty, not eliminate it. Product teams rarely get perfect information. The aim is to gather enough evidence to make a better decision than we would have made from assumptions alone.

I usually look at the problem from several angles. User conversations tell me how people describe the experience. Product data shows me what they actually do. Support tickets reveal recurring frustrations. Stakeholders provide business context. Competitor research can show whether the problem is unique to us or part of a broader market pattern.

The interesting part is when those signals disagree. A user may say one thing while their behaviour suggests another. That is not a reason to choose one source and ignore the rest. It is usually a reason to ask better questions.

The difference between a symptom and a problem

One of the most useful things a Product Manager can do is separate a symptom from a root cause. ‘Users are abandoning checkout’ is a symptom. The actual problem might be an unclear fee, a payment failure, a slow response, too many steps, or a lack of trust at the final moment.

If we treat the symptom as the problem, we can easily build something that looks productive but changes nothing. Discovery gives the team a chance to frame the problem correctly before the cost of changing direction becomes much higher.

This is also where curiosity matters. I ask ‘why?’ repeatedly, but not to be difficult. I ask because the first answer is often the easiest answer, not the most useful one.

How I know discovery has done its job

Discovery is useful when the team can explain the problem in simple language. We should know who is affected, what they are trying to achieve, why the current experience is inadequate and what evidence supports the opportunity.

We may still have open questions. That is normal. What matters is that we are no longer building because somebody had an idea. We are moving forward because we have a reason.

Sometimes the best outcome of discovery is deciding not to build. That can feel less exciting than launching a feature, but avoiding the wrong investment is also product impact.

Discovery teardown: how I break the work down

USER	Who is experiencing the problem and what are they trying to achieve?
EVIDENCE	What do interviews, feedback, analytics and support conversations tell us?
ROOT CAUSE	What is actually creating the friction beneath the visible symptom?
CONTEXT	Why does this matter to the user and to the business now?
ASSUMPTIONS	What are we currently treating as fact without enough evidence?
DECISION	Do we proceed, test further, reframe the problem or deliberately do nothing?

Discovery does not make a Product Manager slower. Done well, it prevents the team from moving quickly in the wrong direction.

Reflection prompts

- What assumption am I most tempted to treat as fact?
- What evidence would change my mind about this problem?
- If we build nothing, what happens to the user?

Product Manager • FinTech • eCommerce • iGaming